AEGES-Q — QML Reproducibility Audit & Controlled Experiments

Target path: `notebooks/05_qml_benchmarks/qml_reproducibility_audit.ipynb`

This notebook establishes a clean evaluation protocol alongside the existing benchmark. It does not overwrite historical notebooks or results.

Audit findings from the existing benchmark

- The preliminary 500-row architecture comparison uses a subset of the official test split, then reuses that same subset in the later calibration-based evaluation. Treat those results as exploratory, not as an untouched final test.
- The larger experiment independently samples validation and test subsets from the official test CSV using different seeds. Different seeds do not guarantee disjoint rows.
- PCA/scaling in the existing benchmark is fitted on the full official training split, which avoids fitting on test rows; this new protocol fits them only on the selected QML training subset.
- F1-only calibration can produce an all-attack classifier. FPR must be reported and any FPR cap must be explicit.

Execution policy: run top-to-bottom. First verify paths, row counts, split integrity, and transformations. Expensive QML training is disabled by default.

```
from __future__ import annotations

import json
import platform
import sys
from pathlib import Path
from importlib.metadata import PackageNotFoundError, version

import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score, confusion_matrix,
    f1_score, precision_score, recall_score, roc_auc_score,
)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# This notebook lives at notebooks/05_qml_benchmarks/; root is two
```

```
levels up.
PROJECT_ROOT = Path.cwd().resolve().parents[1]
TRAIN_PATH = PROJECT_ROOT / 'data' / 'raw' / 'UNSW_NB15_training-
set.csv'
TEST_PATH = PROJECT_ROOT / 'data' / 'raw' / 'UNSW_NB15_testing-
set.csv'
PREPROCESSOR_PATH = PROJECT_ROOT / 'artifacts' / 'classical' /
'variant_e_preprocessor.joblib'
RESULTS_DIR = PROJECT_ROOT / 'artifacts' / 'qml'
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

print('Project root:', PROJECT_ROOT)
print('Python:', sys.version)
print('Platform:', platform.platform())
for package in ['numpy', 'pandas', 'scikit-learn', 'joblib', 'qiskit',
'qiskit-aer']:
    try:
        print(f'{package}: {version(package)}')
    except PackageNotFoundError:
        print(f'{package}: NOT INSTALLED')
```

```
Project root: C:\Projects\Aeges-Q
Python: 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC
v.1944 64 bit (AMD64)]
Platform: Windows-11-10.0.26200-SP0
numpy: 2.5.2
pandas: 3.0.5
scikit-learn: 1.9.0
joblib: 1.5.3
qiskit: 2.5.2
qiskit-aer: 0.17.2
```

```
required_paths = [TRAIN_PATH, TEST_PATH, PREPROCESSOR_PATH]
missing_paths = [str(path) for path in required_paths if not
path.exists()]
if missing_paths:
    raise FileNotFoundError('Required files missing; check
PROJECT_ROOT and paths:\n' + '\n'.join(missing_paths))

import joblib
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)
preprocessor = joblib.load(PREPROCESSOR_PATH)

assert 'label' in train_df.columns and 'label' in test_df.columns
assert len(train_df) == 82_332, f'Unexpected train row count:
{len(train_df)}'
assert len(test_df) == 175_341, f'Unexpected test row count:
{len(test_df)}'
assert train_df.columns.tolist() == test_df.columns.tolist(),
```

```
'Train/test schema mismatch'

print('Train shape:', train_df.shape)
print('Test shape:', test_df.shape)
print('Preprocessor:', type(preprocessor).__name__)
print('Transformed feature count:',
len(preprocessor.get_feature_names_out()))
print('Train label distribution:\n',
train_df['label'].value_counts(normalize=True).sort_index())
print('Test label distribution:\n',
test_df['label'].value_counts(normalize=True).sort_index())
```

```
Train shape: (82332, 45)
Test shape: (175341, 45)
Preprocessor: ColumnTransformer
Transformed feature count: 177
Train label distribution:
 label
0    0.4494
1    0.5506
Name: proportion, dtype: float64
Test label distribution:
 label
0    0.319378
1    0.680622
Name: proportion, dtype: float64
```

1. Create disjoint subsets

Model fitting draws only from the official training CSV. Validation and final test are partitioned once from the official testing CSV, with original CSV row IDs retained. This makes validation/test overlap impossible by construction.

```
QML_TRAIN_SIZE = 2_000
QML_VALIDATION_SIZE = 1_000
QML_TEST_SIZE = 2_000

train_row_ids = np.arange(len(train_df), dtype=np.int64)
test_row_ids = np.arange(len(test_df), dtype=np.int64)
if QML_TRAIN_SIZE > len(train_df):
    raise ValueError('QML_TRAIN_SIZE exceeds official training split')
if QML_VALIDATION_SIZE + QML_TEST_SIZE > len(test_df):
    raise ValueError('Validation + test sizes exceed official testing
split')

train_indices, _ = train_test_split(
    train_row_ids, train_size=QML_TRAIN_SIZE,
    stratify=train_df['label'].to_numpy(), random_state=RANDOM_STATE,
)
```

```
validation_indices, remaining_test_indices = train_test_split(
    test_row_ids, train_size=QML_VALIDATION_SIZE,
    stratify=test_df['label'].to_numpy(), random_state=RANDOM_STATE +
1,
)
final_test_indices, _ = train_test_split(
    remaining_test_indices, train_size=QML_TEST_SIZE,
    stratify=test_df.iloc[remaining_test_indices]['label'].to_numpy(),
    random_state=RANDOM_STATE + 2,
)

# Row indices belong to separate CSV files.
# Only validation and final-test indices must be disjoint,
# because both refer to rows in the official testing CSV.

assert len(validation_indices) == QML_VALIDATION_SIZE
assert len(final_test_indices) == QML_TEST_SIZE

assert set(validation_indices).isdisjoint(
    set(final_test_indices)
)

assert len(train_indices) == QML_TRAIN_SIZE

print("Training rows:", len(train_indices))
print("Validation rows:", len(validation_indices))
print("Final test rows:", len(final_test_indices))
print("Validation/test overlap:", len(
    set(validation_indices) & set(final_test_indices)
))

X_train_raw = train_df.iloc[train_indices].drop(columns=['label'])
y_train = train_df.iloc[train_indices]
['label'].to_numpy(dtype=np.int8)
X_val_raw = test_df.iloc[validation_indices].drop(columns=['label'])
y_val = test_df.iloc[validation_indices]
['label'].to_numpy(dtype=np.int8)
X_test_raw = test_df.iloc[final_test_indices].drop(columns=['label'])
y_test = test_df.iloc[final_test_indices]
['label'].to_numpy(dtype=np.int8)

split_manifest = {
    'random_state': RANDOM_STATE,
    'train_csv_rows': train_indices.tolist(),
    'validation_csv_rows': validation_indices.tolist(),
    'test_csv_rows': final_test_indices.tolist(),
}
manifest_path = RESULTS_DIR / 'qml_split_manifest.json'
with manifest_path.open('w', encoding='utf-8') as file:
    json.dump(split_manifest, file)
```

```
print('Train:', X_train_raw.shape, 'Validation:', X_val_raw.shape,
'Final test:', X_test_raw.shape)
print('Validation/test overlap:',
len(set(validation_indices).intersection(final_test_indices)))
print('Saved split manifest:', manifest_path)
```

```
Training rows: 2000
Validation rows: 1000
Final test rows: 2000
Validation/test overlap: 0
Train: (2000, 44) Validation: (1000, 44) Final test: (2000, 44)
Validation/test overlap: 0
Saved split manifest: C:\Projects\Aeges-Q\artifacts\qml\
qml_split_manifest.json
```

2. Transform features without fitting on validation/test

Reuse the saved Variant E preprocessor as-is. Fit PCA and the angle scaler on the selected QML training subset only. This will not exactly reproduce the old PCA representation, which was fitted on the full official training split; it is a new, explicitly defined protocol.

```
X_train_transformed = preprocessor.transform(X_train_raw)
X_val_transformed = preprocessor.transform(X_val_raw)
X_test_transformed = preprocessor.transform(X_test_raw)

def to_dense_float32(matrix):
    if hasattr(matrix, 'toarray'):
        matrix = matrix.toarray()
    return np.asarray(matrix, dtype=np.float32)

X_train_dense = to_dense_float32(X_train_transformed)
X_val_dense = to_dense_float32(X_val_transformed)
X_test_dense = to_dense_float32(X_test_transformed)

N_QUBITS = 4
pca = PCA(n_components=N_QUBITS, random_state=RANDOM_STATE)
X_train_pca = pca.fit_transform(X_train_dense)
X_val_pca = pca.transform(X_val_dense)
X_test_pca = pca.transform(X_test_dense)

angle_scaler = MinMaxScaler(feature_range=(0.0, 1.0))
X_train_angles = angle_scaler.fit_transform(X_train_pca) * np.pi
X_val_angles = angle_scaler.transform(X_val_pca) * np.pi
X_test_angles = angle_scaler.transform(X_test_pca) * np.pi

assert X_train_angles.shape[1] == N_QUBITS
assert X_val_angles.shape[1] == N_QUBITS
assert X_test_angles.shape[1] == N_QUBITS
```

```
assert np.isfinite(X_train_angles).all()
assert np.isfinite(X_val_angles).all()
assert np.isfinite(X_test_angles).all()

print('PCA explained variance:', pca.explained_variance_ratio_)
print('Total explained variance:',
float(pca.explained_variance_ratio_.sum()))
print('Angle array shapes:', X_train_angles.shape, X_val_angles.shape,
X_test_angles.shape)
print('Train angle range:', float(X_train_angles.min()),
float(X_train_angles.max()))
print('Note: validation/test values can fall outside [0, pi] if
outside training min/max.')
```

```
PCA explained variance: [0.23608711 0.13531207 0.0919031  0.05781537]
Total explained variance: 0.521117627620697
Angle array shapes: (2000, 4) (1000, 4) (2000, 4)
Train angle range: 0.0 3.1415927410125732
Note: validation/test values can fall outside [0, pi] if outside
training min/max.
```

3. Metric and validation-calibration helpers

The threshold/polarity is selected using validation data only. An FPR constraint is explicit. If no candidate meets it, the function raises rather than silently relaxing the constraint. A validation FPR cap is not a guarantee that final-test FPR will satisfy the same cap.

```
def binary_metrics(y_true, probabilities, threshold):
    probabilities = np.asarray(probabilities, dtype=float)
    predictions = (probabilities >= float(threshold)).astype(np.int8)
    tn, fp, fn, tp = confusion_matrix(y_true, predictions, labels=[0,
1]).ravel()
    fpr = fp / (fp + tn) if (fp + tn) else float('nan')
    specificity = tn / (tn + fp) if (tn + fp) else float('nan')
    return {
        'accuracy': accuracy_score(y_true, predictions),
        'precision': precision_score(y_true, predictions,
zero_division=0),
        'recall': recall_score(y_true, predictions, zero_division=0),
        'f1': f1_score(y_true, predictions, zero_division=0),
        'roc_auc': roc_auc_score(y_true, probabilities),
        'fpr': fpr, 'specificity': specificity,
        'balanced_accuracy': balanced_accuracy_score(y_true,
predictions),
        'threshold': float(threshold),
        'tn': int(tn), 'fp': int(fp), 'fn': int(fn), 'tp': int(tp),
    }
```

```
def choose_threshold_on_validation(y_true, raw_probabilities,
max_fpr=0.10):
    """Choose polarity + threshold on validation data only."""
    y_true = np.asarray(y_true)
    raw_probabilities = np.asarray(raw_probabilities, dtype=float)
    candidates = []
    for polarity in ('normal', 'inverted'):
        probs = raw_probabilities if polarity == 'normal' else 1.0 -
raw_probabilities
        unique = np.unique(probs)
        thresholds = np.unique(np.concatenate(([0.0, 1.0], unique)))
        for threshold in thresholds:
            metrics = binary_metrics(y_true, probs, threshold)
            if metrics['fpr'] <= max_fpr:
                candidates.append({'polarity': polarity, **metrics})
    if not candidates:
        raise RuntimeError(f'No threshold satisfies validation FPR <=
{max_fpr:.3f}')
    return max(candidates, key=lambda row: (row['f1'], row['recall'],
-row['fpr']))


def apply_polarity(raw_probabilities, polarity):
    raw_probabilities = np.asarray(raw_probabilities, dtype=float)
    if polarity == 'normal':
        return raw_probabilities
    if polarity == 'inverted':
        return 1.0 - raw_probabilities
    raise ValueError(f'Unknown polarity: {polarity}')
```

4. Existing artifact inventory

Record existing outputs without overwriting them. Historical QML results remain a separate reference.

```
for directory in [PROJECT_ROOT / 'artifacts' / 'classical',
RESULTS_DIR, PROJECT_ROOT / 'results']:
    print(f'\n[{directory}]')
    if directory.exists():
        for path in sorted(directory.glob('*')):
            if path.is_file():
                print(f'  {path.name} ({path.stat().st_size:,}
bytes)')
    else:
        print('  directory not found')

historical_csv = RESULTS_DIR / 'qml_circuit_c_local_results.csv'
if historical_csv.exists():
    print('\nHistorical QML result CSV:')
```

```
    display(pd.read_csv(historical_csv))
else:
    print('\nHistorical QML result CSV not found; this notebook will
not create/overwrite it.')
```

```
[C:\Projects\Aeges-Q\artifacts\classical]
  .gitkeep (0 bytes)
  random_forest.joblib (128,441,769 bytes)
  variant_e_preprocessor.joblib (6,436 bytes)

[C:\Projects\Aeges-Q\artifacts\qml]
  qml_circuit_c_local_results.csv (329 bytes)
  qml_split_manifest.json (35,915 bytes)

[C:\Projects\Aeges-Q\results]
  .gitkeep (0 bytes)

Historical QML result CSV:

           model  qubits  features  training_samples
validation_samples  \
0  QML Circuit C       4         4              2000
1000

   test_samples  accuracy  precision    recall        f1   roc_auc
fpr  \
0          2000     0.677   0.916182  0.578251  0.709009  0.720401
0.112676

   training_time_seconds  inference_time_seconds
0             616.229013                18.01991
```

5. Controlled QML training — intentionally gated

Do not run a costly training loop until the split/integrity checks pass and the circuit implementation is reviewed. The next step is to import/reuse the selected circuit implementation, test probability conventions on simple deterministic circuits, then add fixed-seed repetitions and runtime/circuit metadata. Keep the historical benchmark notebook unchanged.

Recommended experiment metadata: seed, train/validation/test row IDs, qubit count, circuit depth, gate counts, shot count, optimizer and parameters, software versions, wall-clock training/inference times, polarity, threshold, and all test metrics.

```
RUN_QML_TRAINING = False

if RUN_QML_TRAINING:
    raise NotImplementedError(
        'Add and validate the selected circuit training implementation
```

```
before enabling. '
        'Do not overwrite qml_circuit_c_local_results.csv.'
    )
else:
    print('Training is disabled. This notebook currently performs
protocol/data audit only.')
```

```
Training is disabled. This notebook currently performs protocol/data
audit only.
```

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

simulator = AerSimulator()

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

compiled = transpile(qc, simulator, seed_transpiler=42)
result = simulator.run(
    compiled,
    shots=1024,
    seed_simulator=42
).result()

counts = result.get_counts()

print("Qiskit Aer: operational")
print("Shots:", sum(counts.values()))
print("Bell-state measurement counts:", counts)
```

```
Qiskit Aer: operational
Shots: 1024
Bell-state measurement counts: {'00': 521, '11': 503}
```

6. Audit Conclusion and Experiment Decisions

Findings

Area	Finding	Decision
Dataset	Official UNSW-NB15 train/test files and 177-feature preprocessor load correctly	Retain
Feature reduction	Four PCA components explain 45.64% variance in the original benchmark; the new subset-fitted PCA	Record the two protocols separately

Area	Finding	Decision
	explains 52.11%	
Circuit A	Feature-encoding-only architecture; weak preliminary classification results	Keep as a reference baseline
Circuit B	Trainable rotations; preliminary ROC-AUC 0.2660	Do not prioritize further training
Circuit C	Trainable rotations plus entanglement; historical calibrated ROC-AUC 0.7204	Retain as the primary QML candidate
Evaluation split	Preliminary test samples were reused during model comparison; larger experiment used independently sampled subsets from the same test CSV	Use the new disjoint validation/test partition
Threshold calibration	F1-only selection produced an all-attack prediction; FPR-constrained selection improved the operating point	Use validation-only calibration and report test FPR
Reproducibility	Historical result CSV exists; split manifest is now saved	Preserve historical artifacts and save new results separately

Limitations

- The historical results and the new split protocol are not directly interchangeable.
- Four-qubit PCA compression discards information; its impact requires further measurement.
- Finite-shot simulation introduces sampling variability.
- The existing experiments do not establish quantum advantage.
- No physical quantum hardware evaluation has been performed.

Decision

The audit is complete enough to proceed. Do not spend additional time building audit infrastructure.

Next experiment: run a controlled, repeated-seed evaluation of Circuit C using the corrected disjoint split. Keep the architecture fixed, select polarity and threshold using validation data only, evaluate the final test set only after configuration is frozen, and record all repetitions and runtimes. Compare the results against the established Random Forest baseline.

Do not overwrite the historical benchmark CSV. Save the new experiment under a separate filename.